

LangChain + LangGraph

Master Revision Guide

From LLM fundamentals to RAG, Tools, Agents, LangGraph, HITL, Persistence and Multi-Agent Architecture

Personal revision document — designed to be understandable from scratch when revisiting a topic.

How to use this guide: Read the mental model first, then the execution flow, then the code pattern. When stuck, use the “What to remember” boxes rather than rereading every implementation detail.

Learning Philosophy Used Throughout

1. Start with the problem the technology is solving.
2. Understand the manual approach when it clarifies the abstraction.
3. See where the manual approach becomes repetitive or difficult to maintain.
4. Introduce the LangChain/LangGraph abstraction.
5. Implement it in JavaScript/Node.js with Ollama.
6. Understand what happens internally at a high level.
7. Finish with a compact mental model and a reusable code pattern.

Important Environment Choices

Area	Choice
Language	JavaScript / Node.js only
LLM runtime	Ollama
Embedding model used	nomic-embed-text
LLM used during learning	qwen2.5:7b
Vector database	Qdrant Cloud
Application style	Node.js / ES modules
Learning constraint	Free/open-source-first; no Python; no Docker for this learning path

Security note: The original uploaded notes contained a live-looking Qdrant API key and cluster endpoint. The key is intentionally NOT reproduced here. Store credentials only in environment variables and rotate any exposed credential.

Contents

Part I — LLM, Ollama and Tool Calling Foundations

Part II — RAG End-to-End: Loading → Chunking → Embeddings → Qdrant → Retrieval → Generation

Part III — LangChain Core: Prompts, Runnables, LCEL and Composition

Part IV — LangChain Tools and Agents

Part V — LangGraph Core: State, Nodes, Edges, Routing, Loops, Parallelism and Reducers

Part VI — LangGraph + Tools + Agentic RAG

Part VII — Reliability: Errors, Retry, Fallback

Part VIII — Human-in-the-Loop, Checkpointing and Persistence

Part IX — Subgraphs and Multi-Agent Workflows

Part X — Production-Oriented LangGraph Architecture

Part XI — High-Value Mistakes and Corrections

Part XII — Fast Revision Cheat Sheets

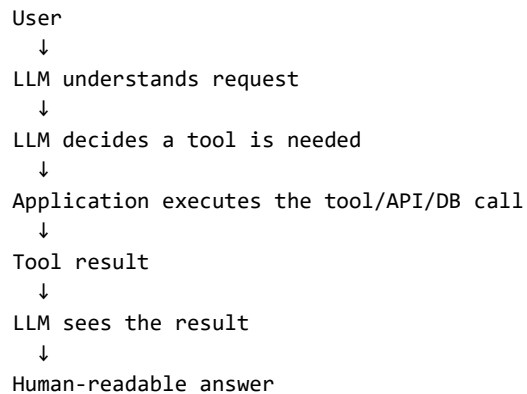
Part XIII — Learning Roadmap After the Series

Part I — LLM, Ollama and Tool Calling Foundations

1. LLM vs Application Code

The LLM is responsible for language reasoning and deciding what should happen next. Application code is responsible for actually performing deterministic work such as calling an API, querying a database, or executing a function.

For tool calling, the clean high-level model is:



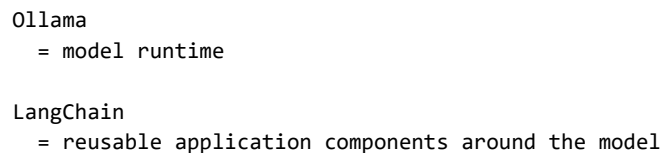
Must remember: The LLM can decide which tool to use, but your application code executes the actual function/API/database operation.

2. Why the LLM may be called twice

8. First model call: understand the user request and decide whether a tool is needed; if needed, emit a tool call.
9. Application executes the requested tool and obtains data.
10. Second model call: give the model the tool result so it can explain or synthesize the final answer.

3. Ollama Mental Model

Ollama is the local model runtime in this learning setup. LangChain does not replace Ollama; it provides standardized abstractions around model invocation, prompts, retrieval, tools and workflows.

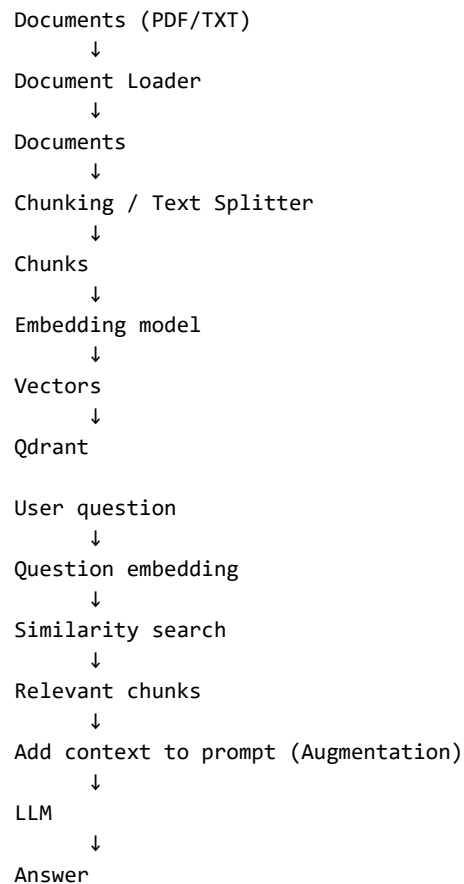


Part II — RAG End-to-End

4. RAG in One Sentence

Definition: RAG (Retrieval-Augmented Generation) means retrieving relevant external knowledge and adding it to the prompt before the LLM generates the answer.

5. The Complete RAG Pipeline



6. Retrieval (R)

The question is embedded, then compared against stored document vectors. The goal is to retrieve the most relevant chunks based on semantic similarity, not exact keyword matches.

7. Augmentation (A)

Augmentation is not merely formatting. The core idea is adding retrieved external context to the user question before generation. Formatting the context and question into a prompt is part of this process.

Context:
Contractors receive 10 leave days annually.

Question:
What is contractor leave policy?

8. Generation (G)

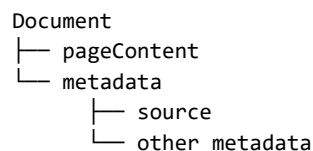
The LLM uses the augmented prompt to generate a natural-language answer.

9. Why not SQL LIKE?

Keyword search fails when users and documents use different words for the same concept. Embeddings represent semantic information, allowing similarity search to find conceptually related text. Retrieving only relevant chunks also reduces the amount of context sent to the LLM.

10. Document Loading

A loader turns files into LangChain Documents. A Document typically contains pageContent plus metadata such as source information.



Must remember: Document loading solves “How do I turn files into structured objects I can process consistently?”

11. Chunking / Text Splitters

Large documents are split into smaller pieces because retrieval should usually return the most relevant portion, not an entire huge document. Chunk size controls the approximate size of each chunk. Chunk overlap preserves some neighboring context across chunk boundaries.

```
Chunk 1: A B C D E F
Chunk 2:      E F G H I J
           ↑ overlap
```

12. RecursiveCharacterTextSplitter Mental Model

RecursiveCharacterTextSplitter tries to split on useful boundaries before falling back to smaller separators. It is a practical general-purpose splitter for prose and mixed text.

Must remember: Chunking is about making retrieval precise and context manageable; overlap is about reducing boundary-context loss.

13. Embeddings

An embedding model converts text into a vector. The document chunk becomes a vector during ingestion; the user question becomes a vector during retrieval.

```
Text chunk
↓
OllamaEmbeddings (nomic-embed-text)
↓
[0.12, -0.42, 0.87, ...]
```

Must remember: The documents are embedded and the query is embedded using a compatible embedding space/model so semantic similarity can be computed.

14. Qdrant

Qdrant stores vectors plus payload metadata and supports vector similarity search. In your learning project the payload includes the chunk text under the “text” field and source information.

```
{
  vector: [...],
  payload: {
    text: "...",
    source: "..."
  }
}
```

15. Vector Store vs Retriever

Component	Purpose
Vector store	Stores/searches vectors and their payloads
Retriever	Presents a retrieval interface that returns relevant Documents
QdrantVectorStore	LangChain abstraction connecting to Qdrant
asRetriever({ k: 3 })	Configures the number of relevant results to retrieve

16. Complete RAG Chain Pattern

```
const ragChain =
  RunnableParallel.from({
    context: retriever,
    question: new RunnablePassthrough()
  })
  .pipe(ragPrompt)
  .pipe(model)
  .pipe(new StringOutputParser());
```

What happens: the same input question fans out into two branches: one branch retrieves context, the other passes the original question through. The combined object is fed into the prompt, then the model, then the parser.

Part III — LangChain Core

17. ChatOllama + invoke()

```
const model = new ChatOllama({
  model: "qwen2.5:7b"
});

const result = await model.invoke("Hello");
```

ChatOllama is the LangChain model abstraction for an Ollama chat model. `invoke()` is the standard execution method used across LangChain runnables.

Must remember: model defines/represents the model capability; `invoke()` actually executes it.

18. PromptTemplate vs ChatPromptTemplate

Prompts turn raw inputs into structured instructions for the model. A prompt template lets you reuse the same instruction structure while inserting variables.

```
const prompt = ChatPromptTemplate.fromTemplate(`
Answer using context:
{context}

Question:
{question}
`);
```

19. Runnable

Definition: A Runnable is something LangChain knows how to execute in a standardized way. It can be invoked and can be composed with other runnables.

```
input
  ↓
Runnable
  ↓
output
```

20. LCEL and .pipe()

LCEL (LangChain Expression Language) is the composition style used to connect runnables into pipelines.

```
prompt
  ↓ .pipe()
model
  ↓ .pipe()
parser
```

The mental model is function composition: output of one runnable becomes the input of the next runnable.

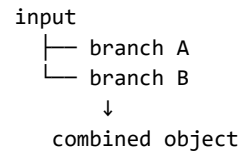
21. RunnablePassthrough

RunnablePassthrough returns the input unchanged. It is useful when part of a parallel pipeline should preserve the original input.

```
RunnableParallel.from({  
  context: retriever,  
  question: new RunnablePassthrough()  
})
```

22. RunnableParallel

RunnableParallel runs multiple independent branches from the same input and combines their outputs into an object.



Must remember: RunnableParallel = fan-out into independent computations; RunnablePassthrough = keep the original input unchanged.

Part IV — Tools and Agents

23. Why Tools Exist

An LLM alone cannot safely or reliably perform every real-world operation. Tools expose capabilities such as weather lookup, database search, API calls or a RAG search function.

24. Defining a Tool

```
const weatherTool = tool(  
  async ({ city }) => ({  
    city,  
    temperature: "30°C",  
    condition: "Sunny"  
  }),  
  {  
    name: "get_weather",  
    description: "Get current weather for a city.",  
    schema: z.object({ city: z.string() })  
  }  
);
```

The Zod schema describes the tool input shape. The description helps the LLM decide when the tool is appropriate.

25. bindTools()

```
const modelWithTools = model.bindTools([  
  weatherTool,  
  timeTool,  
  ragTool  
]);
```

bindTools configures the model with the available tool definitions. This does not itself execute the tools.

26. Tool Call Lifecycle

```
User  
↓  
modelWithTools  
↓  
AIMessage with tool_calls  
↓  
application/tool runtime executes tool  
↓  
ToolMessage  
↓  
model sees result  
↓  
final AIMessage
```

Must remember: Tool Calling = the model decides/requests a tool. The tool runtime executes the requested capability.

27. Multiple Tools

A model can have multiple tools. The model may request one or more tools. Independent calls can be executed in parallel; dependent calls must be sequenced.

28. Tool Registry Mental Model

```
tools = [weatherTool, timeTool, ragTool]

model.bindTools(tools)

ToolNode(tools)
```

29. RAG as a Tool

A RAG chain is a fixed process. Wrapping it as a tool turns that fixed process into an LLM-callable capability.

```
const ragTool = tool(
  async ({ question }) =>
    await ragChain.invoke(question),
  {
    name: "search_knowledge_base",
    description: "Retrieve and answer from stored knowledge.",
    schema: z.object({ question: z.string() })
  }
);
```

30. Agent Mental Model

```
Agent
├─ Model
├─ Tools
├─ State / message history
└─ Execution loop
```

A chain follows a known process. An agent is useful when the LLM should dynamically decide what action/tool to take next.

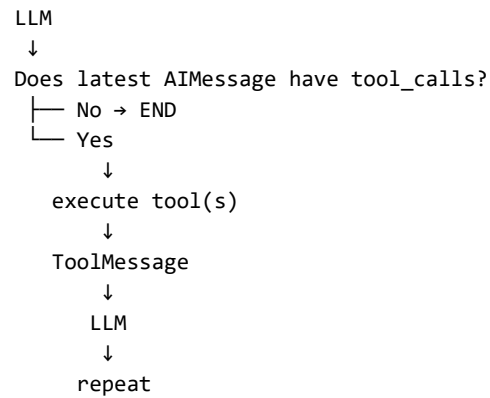
31. createAgent()

```
const agent = createAgent({
  model,
  tools,
  systemPrompt: "...",
});

const result = await agent.invoke({
  messages: [new HumanMessage("...")]
});
```

Construction configures the agent; invoke() runs the agent. The current JS agent API uses createAgent() in the learning path.

32. Agent Loop



Critical stopping rule: The loop continues because the latest AIMessage contains non-empty tool_calls. “Tools exist” or “ToolMessage exists” is NOT the stopping condition.

33. Agent vs Chain vs LangGraph

Abstraction	Best mental model	Control
Chain	Known fixed sequence $A \rightarrow B \rightarrow C$	Application/framework defines path
Agent	LLM decides next action/tool	LLM controls route dynamically
LangGraph	Explicit stateful workflow with controlled routing	Graph controls structure; LLM can still reason inside nodes

Part V — LangGraph Core

34. Why LangGraph Exists

Chains are good when the process is known. Agents are good when the LLM should dynamically choose actions. LangGraph is useful when you need explicit orchestration around stateful, multi-step, branching, looping, parallel or pausable workflows.

```
State
↓
Nodes (work)
↓
Edges (route)
↓
Conditional routing / loops / branches
↓
Stateful workflow
```

35. The Three Core Words

STATE = DATA Shared workflow information carried through the execution.

NODE = WORK Executable work that reads current state and returns a state update.

EDGE = ROUTE Connection that determines where execution goes next.

36. State Schema vs Runtime State

`const State = Annotation.Root({...})` defines the shape/contract. It is not the runtime values. `app.invoke({...})` supplies initial values for one execution. Nodes receive the current runtime state and return updates; LangGraph merges those updates and continues.

```
State definition
  = schema/contract

app.invoke(initialState)
  = initial runtime values

state inside node
  = current runtime state
```

37. Node Pattern

```
const myNode = async (state) => {
  // do work
  return {
    answer: "new value"
  };
};
```

The node normally returns a partial state update, not a full copy of the state.

38. START and END

```
START
↓
first node
↓
...
↓
END
```

39. Conditional Routing

```
.addConditionalEdges(
  "nodeA",
  router,
  {
    yes: "nodeB",
    no: "nodeC"
  }
)
```

The router returns a route key; the mapping sends that key to a destination node.

40. Loops / Cycles

$A \rightarrow B \rightarrow A$

A loop is created by an edge back to an earlier node. State changes such as `retryCount` must eventually satisfy a termination condition.

41. Parallel Execution

```
START
├── Weather
└── Knowledge
    ↓
  Synthesis
    ↓
  END
```

Independent branches can execute in parallel. If multiple branches write the same state key, a reducer may be needed to define how updates merge.

42. Reducers

```
results: Annotation({
  reducer: (existing, update) =>
    [...existing, ...update],
  default: () => []
})
```

Must remember: Reducer = merge rule for multiple updates to the same state key. A reducer is not required just because a graph has parallel branches; it is required when conflicting/shared-key updates need a merge strategy.

Part VI — ToolNode and Agentic RAG

43. ToolNode

ToolNode is a LangGraph prebuilt node that executes tool calls found in the latest AIMessage and produces tool result messages.

```
const toolNode = new ToolNode([
  weatherTool,
  timeTool,
  ragTool
]);
```

Must remember: Tool Calling = model decides/requests the tool. ToolNode = executes the requested tool inside the graph.

44. LLM ↔ ToolNode Loop

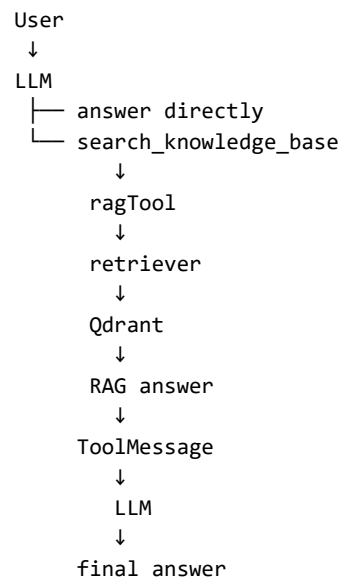
```
START
↓
callModel
↓
shouldContinue
├─ end → END
└─ tools → ToolNode
    ↓
    callModel
    ↓
    ...
```

```
const callModel = async (state) => {
  const response = await modelWithTools.invoke(state.messages);
  return { messages: [response] };
};
```

```
const shouldContinue = (state) => {
  const last = state.messages[state.messages.length - 1];
  return last.tool_calls?.length ? "tools" : "end";
};
```

45. Agentic RAG

Agentic RAG means the agent can decide when the knowledge base capability is needed, rather than every query being forced through RAG.



This keeps the distinction clear: RAG chain = fixed retrieval-and-generation process; RAG tool = exposes that capability to the agent; ToolNode = executes it; LangGraph = orchestrates the loop.

Part VII — Reliability: Error Handling, Retry and Fallback

46. Error Handling

An error can be handled locally with try/catch or represented in graph State when the workflow needs to route based on the failure.

```
try {  
  // operation  
} catch (error) {  
  return {  
    error: error.message  
  };  
}
```

47. Graph-Level Error Routing

```
operation  
↓  
state.error?  
├─ no → success  
└─ yes → error route
```

48. Retry

Definition: Retry means trying the same failed operation again, usually only for transient failures and with a maximum attempt count.

```
attempt 1 ❌  
↓  
attempt 2 ❌  
↓  
attempt 3 ✅  
↓  
continue
```

Retry in a graph is a conditional loop: operation → error state → router → same operation again, with a changing retryCount and a termination condition.

49. Retry is Not Correct for Every Error

Failure	Retry?	Why
Network timeout	Usually yes	Potentially transient
Temporary 503	Usually yes	Potentially transient
Rate limit	Sometimes, with backoff	Temporary but needs policy
Invalid API key	No	Configuration/auth problem
Collection does not exist	No	Configuration/data problem
Invalid user input	No	Same request will keep failing

50. Fallback

Definition: Fallback means switching to a different strategy after the preferred strategy is unavailable or retries are exhausted.

```
Primary RAG
  ↓ fail
retry × 6
  ↓ still failing
Fallback
  ↓
normal LLM / cache / alternate source / human escalation
```

A fallback must be semantically safe. If the primary knowledge source is authoritative and unavailable, a general LLM answer should not silently look like a verified answer.

51. Qdrant-Specific Learning Point

A Qdrant connection created at module startup can fail before the graph begins. If you want the graph itself to catch and route a Qdrant failure, put the failure-prone operation inside an executable node/tool boundary that the graph can observe.

```
Program startup
  ↓
Qdrant connection attempt
  ↓
CRASH before graph
```

versus

```
Graph starts
  ↓
RAG node
  ↓
Qdrant failure
  ↓
state.error
  ↓
retry/fallback
```

Part VIII — Human-in-the-Loop, Checkpointing and Persistence

52. Human-in-the-Loop (HITL)

Simple definition: AI performs some work → pauses → human gives a decision → workflow continues.

```
START
↓
prepare
↓
humanApproval
↓
interrupt()
↓
PAUSE 
↓
Human says YES / NO
↓
resume
↓
continue / reject
↓
END
```

HITL should be used when human judgment is important: booking, payment, deletion, deployment, approval, or other consequential actions.

53. interrupt()

interrupt() pauses the workflow and asks for external input. It does not mean “return a value immediately.” Think “pause now; receive a resume value later.”

54. Why the First HITL Example Looked Like Two Invocations

The two app.invoke() calls represent two different moments. First invocation starts the workflow and pauses at interrupt(). A later application event invokes the same graph with Command({ resume: value }) to continue the paused thread.

```
FIRST MOMENT
app.invoke(initialState, config)
↓
interrupt()
↓
PAUSED

LATER EVENT
human clicks YES
↓
app.invoke(Command({ resume: true }), same config)
↓
workflow resumes
```

Critical mental model: The second app.invoke() is NOT automatic. Your application calls it after it receives the human response.

55. pausedResult Is Not the Resume Mechanism

A variable such as `pausedResult` simply stores the return/control information from the first invocation. It is not required to resume the workflow. The connection is the checkpoint plus the same `thread_id` plus the resume Command.

56. Checkpointing

Definition: Checkpointing means saving snapshots of workflow state/execution so the workflow can be resumed or recovered.

```
interrupt()
  ↓
checkpoint
  ↓
pause
  ↓
resume
  ↓
continue
```

57. MemorySaver

`MemorySaver` is a simple in-memory checkpoint store suitable for learning/testing. Its main limitation is process lifetime: if the Node.js process dies, in-memory state is lost.

```
const checkpointer = new MemorySaver();
const app = graph.compile({ checkpointer });
```

58. thread_id

`thread_id` identifies a particular workflow execution/conversation so the runtime knows which saved checkpoint belongs to which paused or ongoing workflow.

```
const config = {
  configurable: {
    thread_id: "flight-booking-001"
  }
};
```

59. Persistence

Definition: Persistence means storing checkpoints durably so workflow state can survive process restarts and long pauses.

```
Checkpointing
  = save workflow execution
```

```
Persistence
  = save it durably
```

```
MemorySaver
  = in-memory implementation
```

Do not conflate: `interrupt()` = pause. Checkpoint = remember. Persistence = durable storage. `thread_id` = identify the workflow. Resume = continue.

60. Minimal HITL Flow to Remember

```
const humanApproval = async (state) => {
  const decision = interrupt({
    message: "Do you approve?",
    request: state.message
  });

  return { approved: decision === true };
};

const checkpointer = new MemorySaver();
const app = graph.compile({ checkpointer });

const config = {
  configurable: { thread_id: "demo-001" }
};

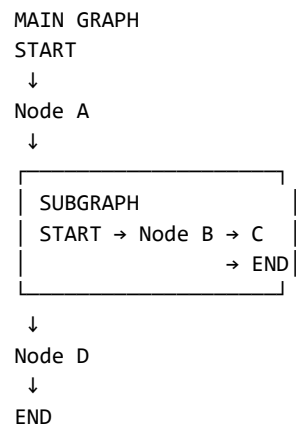
await app.invoke(initialState, config);

// Later, after the human responds:
await app.invoke(
  new Command({ resume: true }),
  config
);
```

Part IX — Subgraphs and Multi-Agent Workflows

61. Subgraphs

Definition: A subgraph is a smaller LangGraph workflow embedded inside a larger LangGraph workflow.



A compiled subgraph can be used as executable work inside a parent graph. It is best thought of as a reusable mini-workflow, not as an AIMessage.

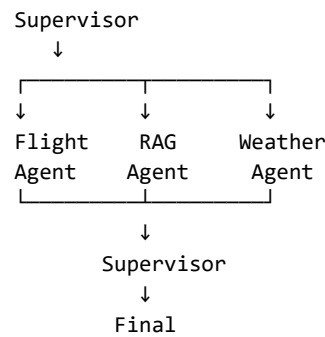
Important correction: A node does not only accept chains, and a subgraph is not “a node because it returns an AIMessage.” A compiled subgraph is executable work. Its internal nodes may produce AI messages, but the compiled subgraph itself is an executable workflow component.

62. Node vs Subgraph

Concept	Mental model
Node	One meaningful unit of executable work
Subgraph	Multiple nodes + edges forming a reusable workflow
Compiled subgraph	Executable workflow component that can be embedded in a parent graph

63. Multi-Agent Workflows

Definition: Multi-agent means multiple specialized reasoning components/agents cooperate through a workflow, usually using shared state, routing, or a supervisor.



64. One Agent vs Multi-Agent

Architecture	Use when
One agent + many tools	One reasoning loop can cleanly handle the responsibilities
Multi-agent	Responsibilities, tools, prompts, policies or workflows benefit from specialization

65. Multi-Agent Is an Architecture Pattern

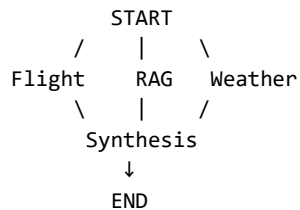
There is not a single “multi-agent node type.” Multi-agent systems can be built using Agents, Nodes, Subgraphs, State, Conditional Routing, Parallel Branches and supervisor patterns.

66. Sequential vs Parallel Multi-Agent

Sequential:

Flight Agent → RAG Agent → Weather Agent → Final

Parallel:



Part X — Production-Oriented LangGraph Architecture

Production-oriented LangGraph is not one new API. It is the discipline of structuring the workflow so it remains understandable, safe, observable and maintainable.

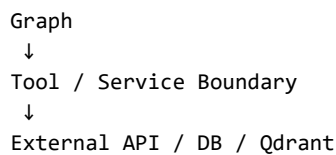
67. Keep Nodes Focused

Prefer small nodes with one meaningful responsibility. Avoid giant nodes that call multiple APIs, perform business rules, call the LLM, update multiple unrelated state fields and send notifications all at once.

68. Keep Routing Simple and Deterministic

Routers should answer “where next?” rather than contain large business algorithms. Use deterministic code for conditions such as `retryCount`, approved flags and required fields.

69. Isolate Tools and External Systems



This separation makes testing, replacement and failure handling easier.

70. Keep State About the Workflow

State should carry the information required between workflow steps, not the entire application infrastructure such as clients, loggers and database connections.

71. Control Loops Explicitly

Every loop needs a termination condition. For retry loops, use `retryCount` and a maximum. For agent loops, define safe limits and termination behavior.

72. Retry Only Transient Errors

Classify the error before retrying when possible. Do not burn time retrying invalid credentials or missing collections.

73. Fallback Must Be Honest

If the primary source is authoritative and unavailable, clearly communicate that the fallback answer is not verified against that source.

74. Use HITL at Meaningful Boundaries

Human approval is valuable where an incorrect automated action has real consequences. It should not be inserted into every small step just because the mechanism exists.

75. Persistence Should Match Workflow Lifetime

Short-lived workflows can sometimes use simple checkpointing. Long-running workflows involving hours/days of pause need durable persistence.

76. Observability

You should be able to identify thread, node, tool, attempt count, error, latency and major state transitions. Keep logging/observability concerns separate from the business data carried in State.

77. LangGraph Is the Orchestration Layer

```
Application
├─ API layer
├─ business services
├─ tools / adapters
├─ LangGraph workflow
├─ persistence
└─ observability
```

Must remember: LangGraph orchestrates a workflow; it is not your entire application architecture.

Part XI — High-Value Mistakes and Corrections

78. Node Definition ≠ Execution Result

Highest-value correction: A LangGraph node should be executable work, not something that has already been executed.

✓ Correct

```
const fallbackNode = async (state) => {
  const response = await model.invoke(...);
  return { answer: response.content };
};

.addNode("fallback", fallbackNode);
```

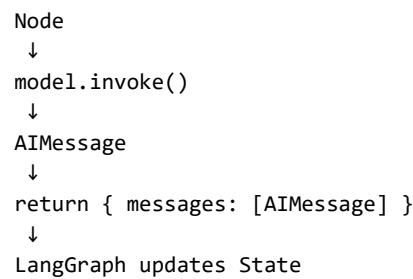
✗ Wrong

```
const fallbackResponse = await model.invoke(...);
.addNode("fallback", fallbackResponse);
```

Why the second version fails: `fallbackResponse` is the result of execution (typically an `AIMessage`), not the executable operation LangGraph should run later.

79. AIMessage Can Exist Inside a Node

The rule is not “nodes cannot have `AIMessage`.” A node can invoke an LLM and receive an `AIMessage`. The node should then return an appropriate state update.



80. Node Inputs and Accepted Executable Forms

At a practical level, `addNode` expects executable work such as a function/async function, `Runnable`, or compiled subgraph. The common error is passing the result of running one of those things instead.

What you pass	Meaning	Typical result
async function(state)	Executable node definition	✓ Valid
Runnable	Executable LangChain unit	✓ Valid
compiled subgraph	Executable mini-workflow	✓ Valid
AIMessage from await model.invoke()	Already-executed result	✗ Not a node definition
string/API result	Already-computed data	✗ Not a node definition

81. Graph Construction vs Graph Execution

```
BUILD PHASE
State schema
↓
node definitions
↓
edges
↓
compile()

EXECUTION PHASE
app.invoke()
↓
START
↓
nodes / routers / tools
↓
END
```

82. HITL Two-Invocation Confusion

The first invoke starts and can pause. A later invoke is explicitly triggered by the application after the human response. There is no automatic “finalResult starts because the variable exists.”

83. Qdrant Startup Failure vs Graph Error Handling

If Qdrant is initialized at module load and the connection fails, the graph may never start. To demonstrate graph-level retry/fallback, keep the failure-prone call inside the node/tool boundary where the graph can catch the error and route based on State.

84. Retry vs Fallback

```
Retry    = same operation again
Fallback = different strategy
```

85. Conditional Routing vs HITL

```
Conditional routing
= choose route using existing State
```

```
HITL
= pause and obtain an external human decision
```

Part XII — Fast Revision Cheat Sheets

86. LangChain One-Page Mental Model

MODEL
ChatOllama
↓ invoke()

PROMPT
PromptTemplate / ChatPromptTemplate

RUNNABLES
Runnable + .pipe() + RunnableParallel + RunnablePassthrough

RAG
Loader → Splitter → Embedding → Vector Store → Retriever
→ Prompt + Context → LLM → Parser

TOOLS
tool() + Zod → bindTools() → tool calls → ToolMessage

AGENT
Model + Tools + State + Loop

87. RAG Cheat Sheet

Stage	Question it answers	Core component
Load	How do I read files?	Document Loader
Split	How do I make retrieval units?	Text Splitter
Embed	How do I represent meaning numerically?	OllamaEmbeddings
Store	Where do vectors live/search?	QdrantVectorStore
Retrieve	How do I get relevant Documents?	Retriever
Augment	How do I add external knowledge?	Prompt + Context
Generate	How do I produce the answer?	ChatOllama / LLM
Parse	How do I normalize the model output?	StringOutputParser

88. Tool Calling Cheat Sheet

```
Tool definition
↓
bindTools()
↓
LLM emits tool_calls
↓
Tool runtime / ToolNode executes
↓
ToolMessage
↓
LLM decides what to do next
```

89. LangGraph Cheat Sheet

```
State      = DATA
Node       = WORK
Edge       = ROUTE
Conditional = DECIDE NEXT ROUTE
Cycle      = LOOP BACK
Parallel   = INDEPENDENT BRANCHES
Reducer    = MERGE RULE
ToolNode   = EXECUTE TOOL CALLS
Subgraph   = MINI-WORKFLOW
Interrupt  = PAUSE
Checkpoint = REMEMBER
Persistence = STORE DURABLY
Multi-agent = SPECIALIZED REASONING COMPONENTS
```

90. The Most Important Execution Rule

Remember forever: Node = executable work. Execution result ≠ node. AIMessage is a possible result produced during node execution.

91. The HITL Rule

Remember forever: interrupt() = pause; checkpoint = remember; same thread_id = identify the workflow; Command({ resume: value }) = continue.

92. The Reliability Rule

Remember forever: Error handling detects/contains failure. Retry repeats the same operation. Fallback switches strategy.

93. Chain vs Agent vs LangGraph

CHAIN

Known path

A → B → C

AGENT

LLM chooses next action

LLM → Tool → LLM → ...

LANGGRAPH

Explicit stateful orchestration

State + Nodes + Edges + routing + loops + pause/resume

Part XIII — Where the Learning Journey Goes Next

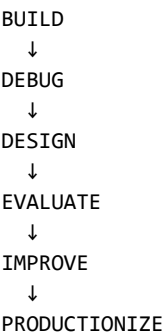
94. Immediate Next Phase

The planned next step after completing the LangGraph concepts is a quick review of the LangChain concepts, especially embeddings, chunking, Qdrant, retrieval, prompts, runnables, tools and agents. The goal is not to relearn every API but to rebuild the connected mental model.

95. MCP Phase

After the review, the next technology topic planned is MCP (Model Context Protocol). The learning approach should start from fundamentals and map MCP concepts against what is already understood about tools, agents and application/model boundaries.

96. Engineering Phase



This phase turns the conceptual knowledge into practical engineering ability. The emphasis moves from “What is this?” to “How do I build, test, measure, improve and operate it?”

97. Personal Progress Snapshot

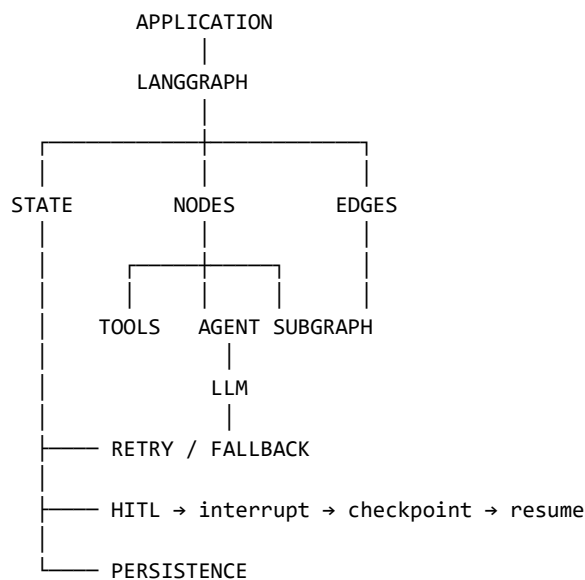
Area	Progress
LLM + Ollama fundamentals	Completed
Embeddings / vector search	Completed
Manual RAG	Completed
LangChain fundamentals	Completed
LangChain tools / tool calling	Completed
LangChain agents	Completed
LangGraph core	Completed
LangGraph advanced workflow concepts	Completed
LangGraph reliability / HITL / persistence	Completed
Subgraphs / multi-agent	Completed
MCP	Next major learning phase

Area	Progress
Build → Debug → Design → Evaluate → Improve → Productionize	After MCP

98. Final “Am I Stuck?” Checklist

11. Am I confusing a definition with an execution result?
12. What is the current runtime State at this exact point?
13. Which node is doing the work?
14. Which edge decides where we go next?
15. Is the operation deterministic or should the LLM decide?
16. Is this a Chain, Tool, Agent, or LangGraph responsibility?
17. If something failed, is it transient (retry) or fundamental (fallback/fix)?
18. If HITL is involved, where is the workflow paused and which thread is being resumed?
19. If parallel branches write the same key, what reducer merges them?
20. If the workflow is large, should a subgraph isolate a responsibility?

99. Final Mental Picture



Final anchor: When you forget a framework API, return to the architecture: what data is flowing, what work is being executed, who decides the next step, and where is the state stored?